# Lecture 21: PINNs Applications, DeepONets and Neural Operators

Sergei V. Kalinin

# Making models

| Type of Error | **Definition** | **Causes** | **How to Reduce** |
|---|---|---|---|
| **Approximation Error** | Difference between best possible function in model class and the true function $f$ | Limited model capacity, wrong architecture | Increase network size, change activation functions, improve architecture |
| **Optimization Error** | Difference between best possible function and what is learned during training | Poor optimizer, bad learning rate, local minima, bad initialization | Use Adam/SGD, adjust learning rate, batch norm, weight initialization |
| **Generalization Error** | Difference between training error and test error | Overfitting, small dataset, high variance | Regularization, more data, dropout, early stopping |

# Green's Functions

A **Green's function** is a fundamental solution used to solve **linear differential equations** with boundary conditions. It represents the **response of a system to a point source**, similar to how the **impulse response** works in signal processing.

Mathematically, for a **linear differential operator** L, the Green's function $G(x,x')$ satisfies:

$$LG(x,x')=\delta(x-x'),$$

where $\delta(x-x')$ is the **Dirac delta function**, representing a **unit impulse** at $x'$.

$G(x,x')$ is known, it can be used to **solve inhomogeneous differential equations** $Lu(x)=f(x)$ by integrating:

$$u(x)=\int G(x,x')f(x')dx'u(x)$$

Thus, Green's functions allow us to express the solution as a **superposition of responses to point sources**.

# Green's Function in Electrostatics

## Green's Function:

$$L \circ u(x) = f(x)$$

$$L \circ G(x,s) = \delta(x-s)$$

$$u(x) = \int G(x,s) f(s) ds$$

## An Example:

$$\nabla^2 \varphi(\vec{r}) = -\frac{\rho(\vec{r})}{\varepsilon_0}$$

$$G(\vec{r},\vec{s}) = \frac{1}{4\pi\varepsilon_0} \frac{1}{|\vec{r}-\vec{s}|}$$

$$\varphi(\vec{r}) = \int G(\vec{r},\vec{s}) \rho(\vec{s}) d^3 s$$

https://galileo-unbound.blog/tag/greens-function/

# Green's Functions depend on D and Domains

**Table 10.1** Fundamental Green's Functions[a]

| | Laplace $\nabla^2$ | Helmholtz[b] $\nabla^2 + k^2$ | Modified Helmholtz[c] $\nabla^2 - k^2$ |
|---|---|---|---|
| 1-D | $\frac{1}{2}\lvert x_1 - x_2\rvert$ | $-\frac{i}{2k}\exp(ik\lvert x_1 - x_2\rvert)$ | $-\frac{1}{2k}\exp(-k\lvert x_1 - x_2\rvert)$ |
| 2-D | $\frac{1}{2\pi}\ln\lvert\boldsymbol{\rho}_1 - \boldsymbol{\rho}_2\rvert$ | $-\frac{i}{4}H_0^{(1)}(k\lvert\boldsymbol{\rho}_1 - \boldsymbol{\rho}_2\rvert)$ | $-\frac{1}{2\pi}K_0(k\lvert\boldsymbol{\rho}_1 - \boldsymbol{\rho}_2\rvert)$ |
| 3-D | $-\frac{1}{4\pi}\frac{1}{\lvert\mathbf{r}_1 - \mathbf{r}_2\rvert}$ | $-\frac{\exp(ik\lvert\mathbf{r}_1 - \mathbf{r}_2\rvert)}{4\pi\lvert\mathbf{r}_1 - \mathbf{r}_2\rvert}$ | $-\frac{\exp(-k\lvert\mathbf{r}_1 - \mathbf{r}_2\rvert)}{4\pi\lvert\mathbf{r}_1 - \mathbf{r}_2\rvert}$ |

# General Features of Green's Functions

1 Linearity – The differential equation must be linear since Green's functions rely on the superposition principle.

2 Specified Differential Operator – The equation must be in the form $Lu(x)=f(x)$ where $L$L is a known linear operator.

3 Specified Domain – The Green's function depends on the shape and size of the domain (e.g., infinite space, bounded region, irregular geometries).

4 Specified Boundary Conditions – The solution is uniquely determined only when boundary conditions (Dirichlet, Neumann, or mixed) are clearly defined.

5 Existence of Fundamental Solution – The Green's function must satisfy $LG(x,x')=\delta(x-x')$ and should be well-defined in the given domain.

6 Integral Representation of the Solution – The final solution must be expressible as an integral of the Green's function weighted by the source term

7 Solvability – In simple cases, Green's functions are found analytically; for complex domains, numerical techniques (e.g., finite elements, boundary elements) may be required.

# Neural Operators

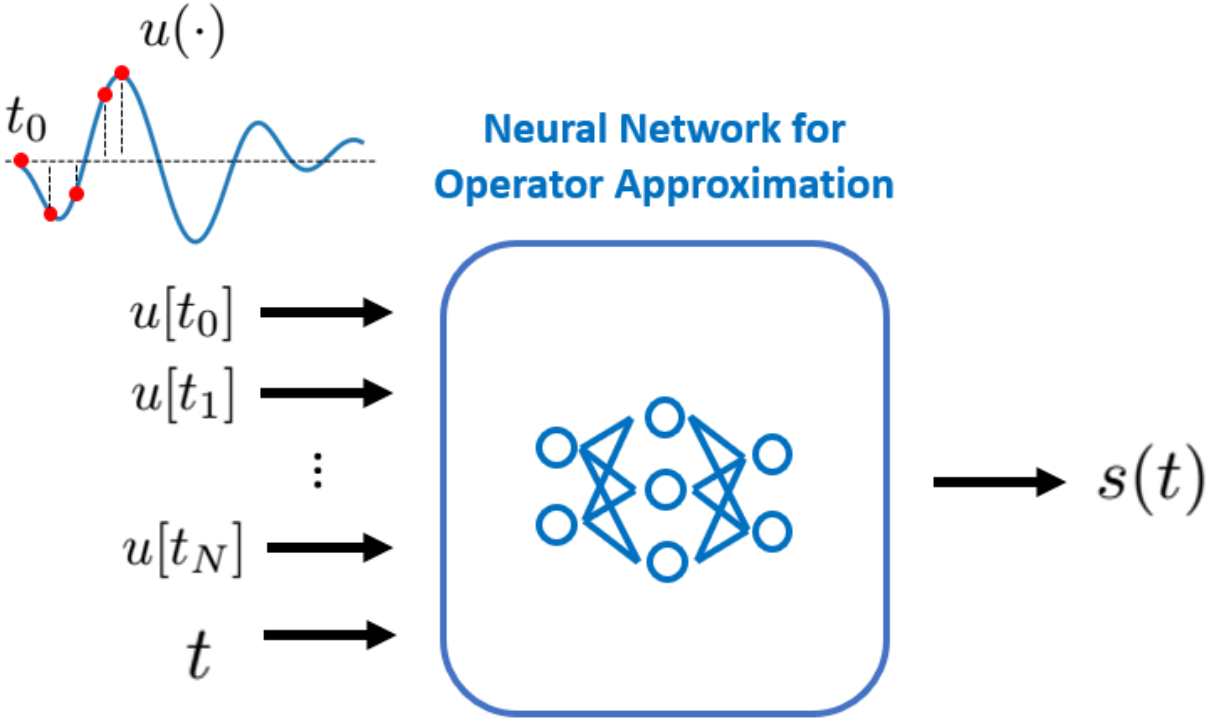
- **Nonlinear PDEs** – Green's functions require linearity

- **Variable Boundary Conditions** – Green's functions require a new derivation for each set of conditions

- **Irregular and Complex Geometries** – Green's functions are often limited to simple shapes

- **High-Dimensional Problems** – Green's functions are difficult to compute for **3D and high-dimensional PDEs**

- **Hybrid Data-Driven and Physics-Based Learning** – Green's functions need explicit physics




https://towardsdatascience.com/solving-inverse-problems-with-physics-informed-deeponet-a-practical-guide-with-code-implementation-27795eb4f502/

# Universal Approximation Theorem

**Theorem 1 (Universal Approximation Theorem for Operator).** *Suppose that $\sigma$ is a continuous non-polynomial function, $X$ is a Banach Space, $K_1 \subset X$, $K_2 \subset \mathbb{R}^d$ are two compact sets in $X$ and $\mathbb{R}^d$, respectively, $V$ is a compact set in $C(K_1)$, $G$ is a nonlinear continuous operator, which maps $V$ into $C(K_2)$. Then for any $\epsilon > 0$, there are positive integers $n$, $p$, $m$, constants $c_i^k, \xi_{ij}^k, \theta_i^k, \zeta_k \in \mathbb{R}$, $w_k \in \mathbb{R}^d$, $x_j \in K_1$, $i = 1, \ldots, n$, $k = 1, \ldots, p$, $j = 1, \ldots, m$, such that*

$$\left| G(u)(y) - \sum_{k=1}^{p} \underbrace{\sum_{i=1}^{n} c_i^k \sigma \left( \sum_{j=1}^{m} \xi_{ij}^k u(x_j) + \theta_i^k \right)}_{branch} \underbrace{\sigma(w_k \cdot y + \zeta_k)}_{trunk} \right| < \epsilon \tag{1}$$

*holds for all $u \in V$ and $y \in K_2$.*

arXiv > cs > arXiv:1910.03193

Search...

Help | Adv

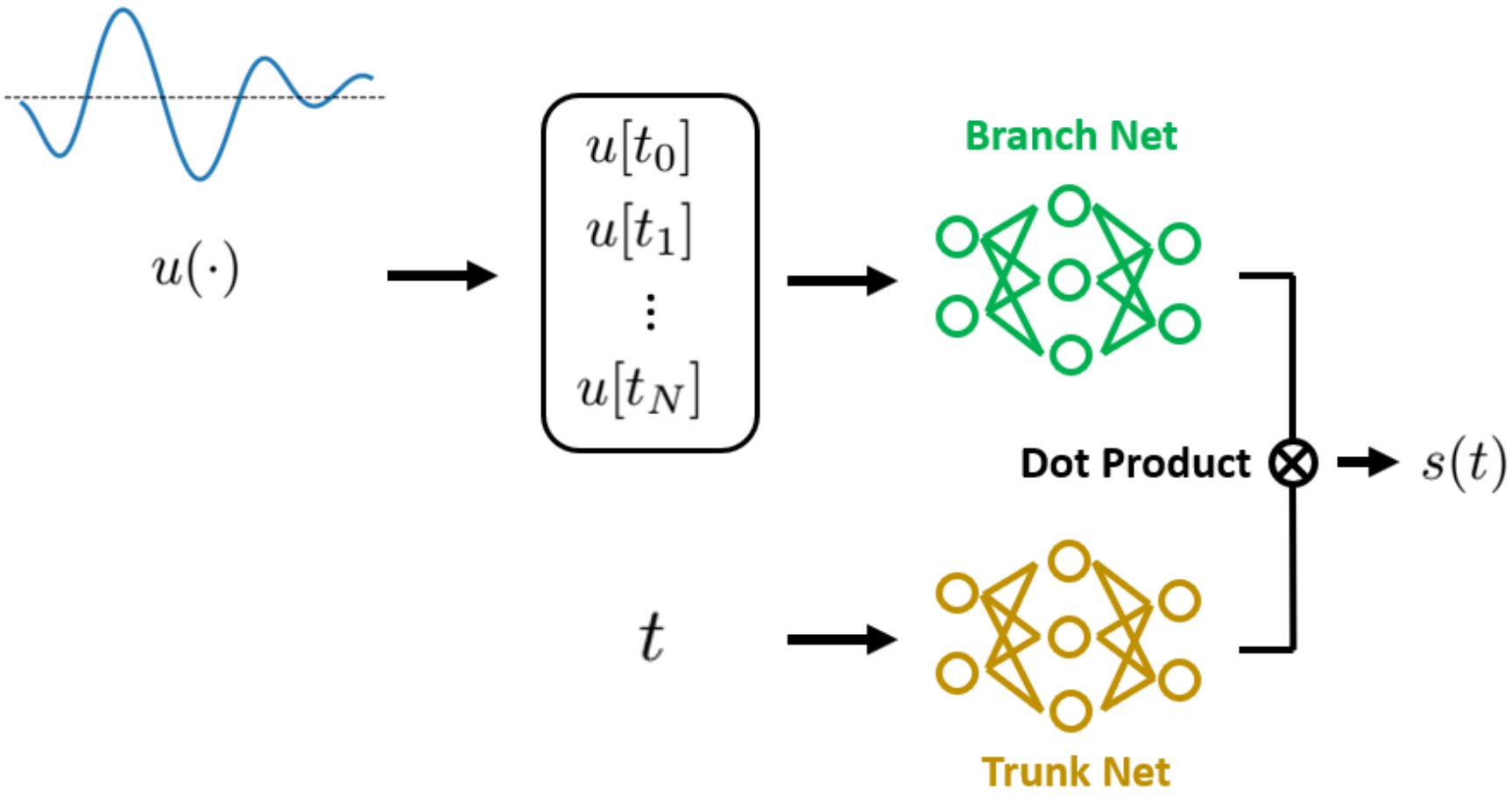
**Computer Science > Machine Learning**

*[Submitted on 8 Oct 2019 (v1), last revised 15 Apr 2020 (this version, v3)]*

**DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators**

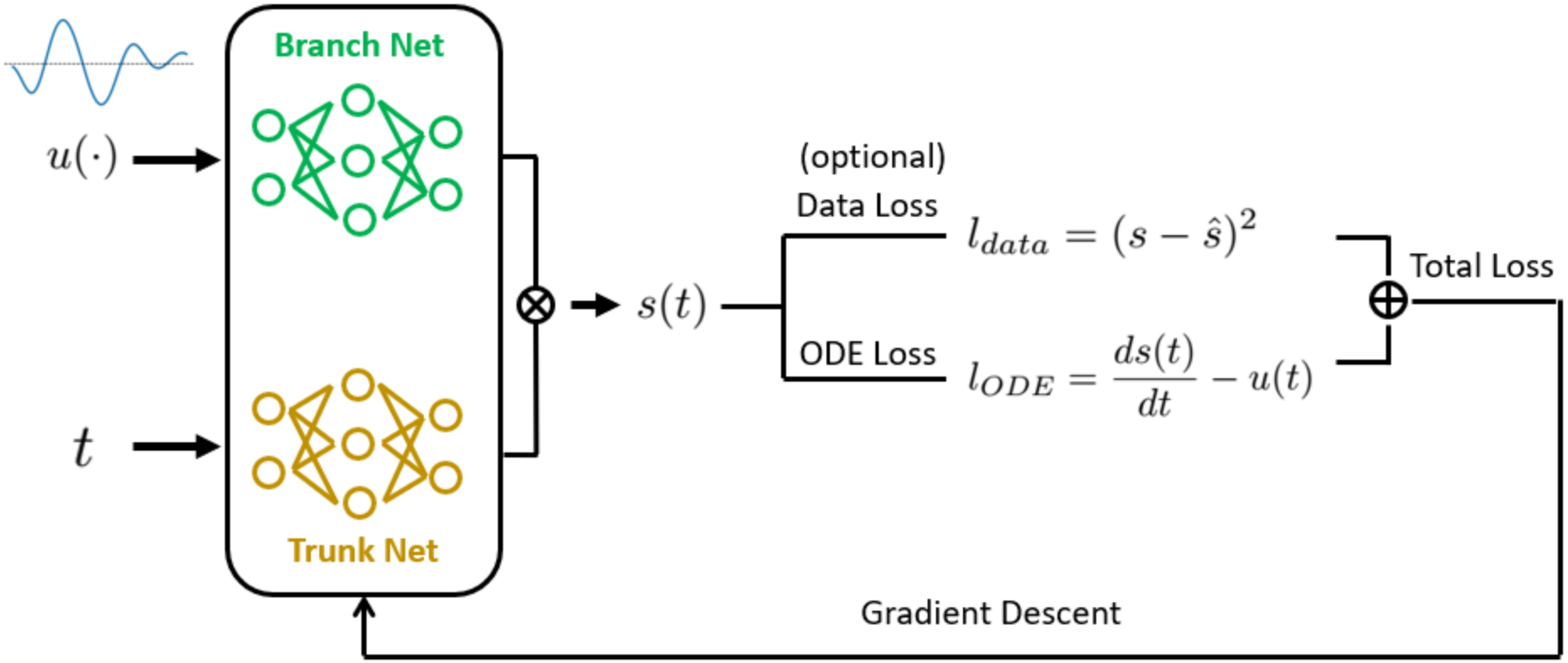
Lu Lu, Pengzhan Jin, George Em Karniadakis

# DeepONet



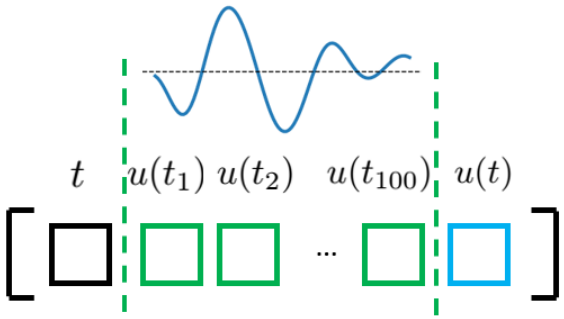

https://towardsdatascience.com/solving-inverse-problems-with-physics-informed-deeponet-a-practical-guide-with-code-implementation-27795eb4f502/

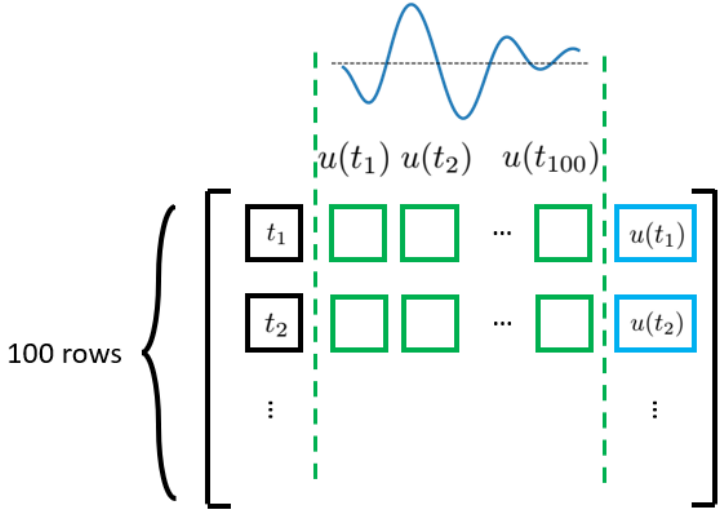
# Physics-Informed DeepONet

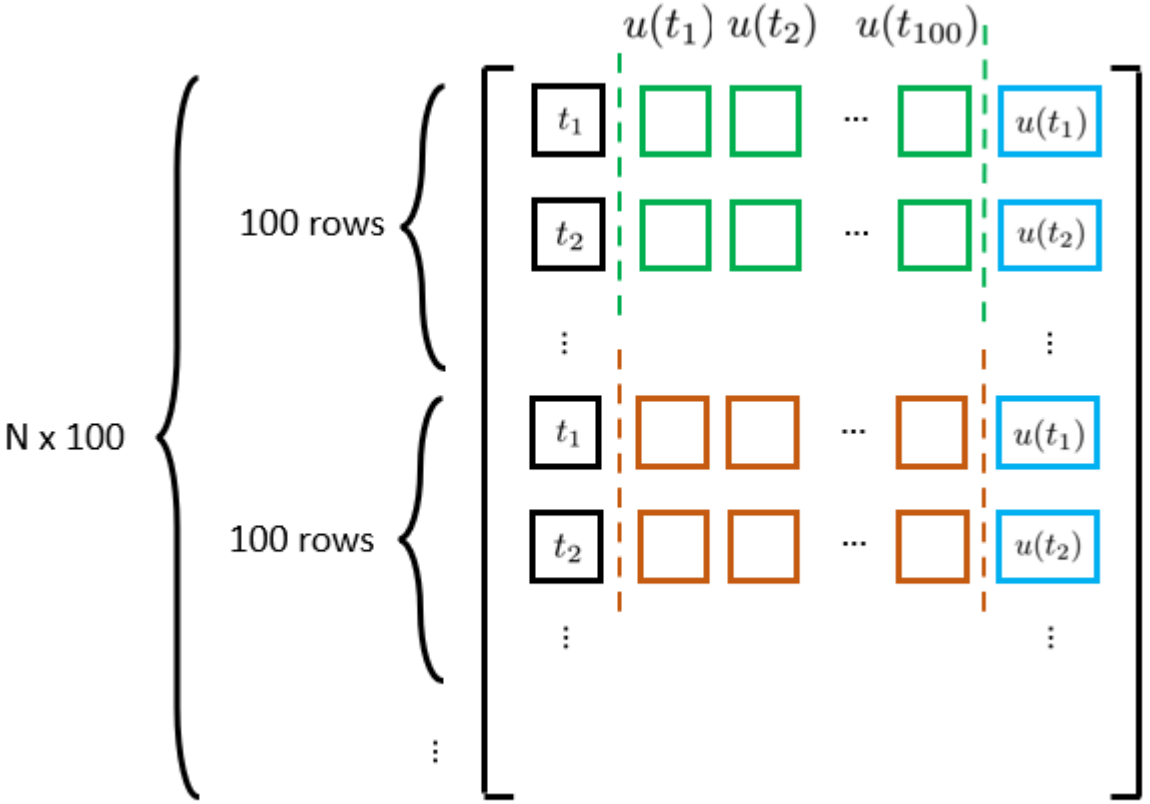



https://towardsdatascience.com/solving-inverse-problems-with-physics-informed-deeponet-a-practical-guide-with-code-implementation-27795eb4f502/

# DeepONet Training

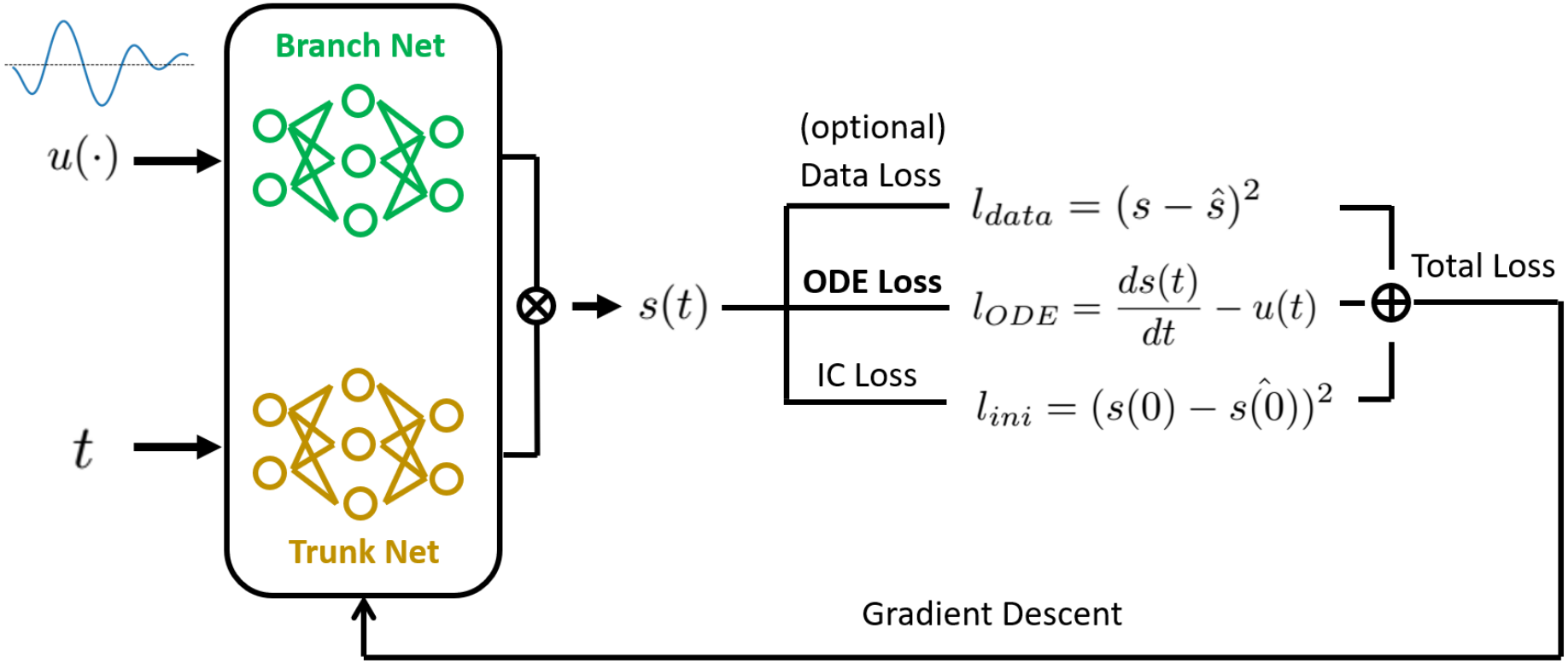



https://towardsdatascience.com/solving-inverse-problems-with-physics-informed-deeponet-a-practical-guide-with-code-implementation-27795eb4f502/

# Physics-Informed DeepONet




https://towardsdatascience.com/solving-inverse-problems-with-physics-informed-deeponet-a-practical-guide-with-code-implementation-27795eb4f502/

# Physics-Informed DeepONet Applications

## Parameter discovery

$$\frac{ds(t)}{dt} = a \cdot u(t) + b, \ t \in [0, 1]$$

s(0) = 0, a and b are **unknowns**. Our objective here is to estimate the values of *a* and *b*, given the observed u(·) and s(·) profiles.

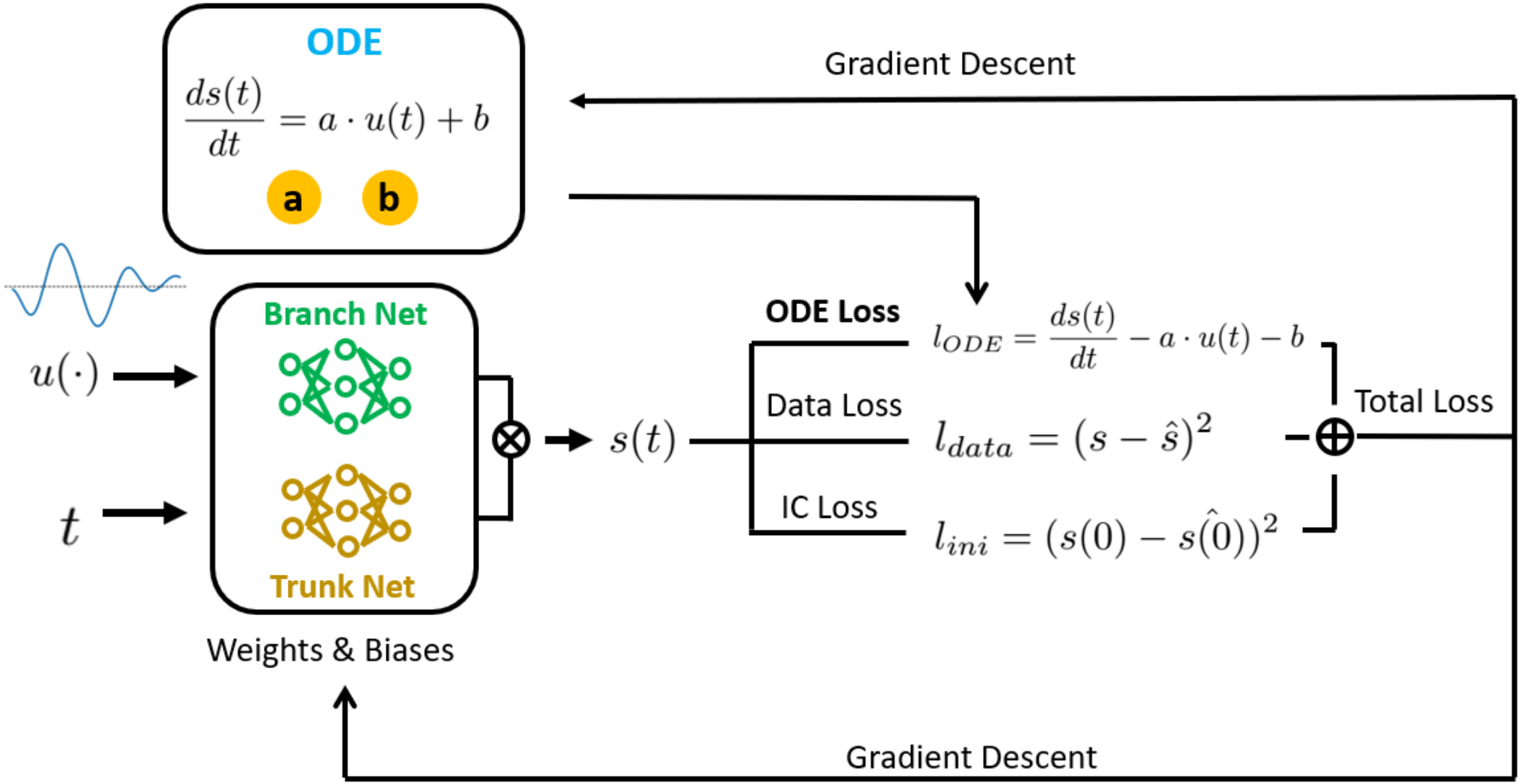
## Function discovery

$$\frac{ds(t)}{dt} = 0.5 \cdot u(t) + 0.5, \ t \in [0, 1]$$

initial condition s(0) = 0, a=0.5, b=0.5. We aim to learn u(t)

https://towardsdatascience.com/solving-inverse-problems-with-physics-informed-deeponet-a-practical-guide-with-code-implementation-27795eb4f502/

# Physics-Informed DeepONet Applications




https://towardsdatascience.com/solving-inverse-problems-with-physics-informed-deeponet-a-practical-guide-with-code-implementation-27795eb4f502/

# Green's Functions vs. PI-DeepONets

• **Analytical Exact Solutions** – Green's functions provide closed-form solutions, while DeepONets are approximations.
• **Infinite Domains** – Green's functions can naturally handle infinite space problems, while DeepONets require **training on finite domains**.
• **Singular Solutions** – Green's functions handle **point singularities** well, while DeepONets may struggle with sharp features.
• **Universal Applicability** – Green's functions work for any system that satisfies the linear PDE, whereas DeepONets must be trained for each problem.
• **Lack of Data Dependency** – Green's functions can be derived directly from physics, while DeepONets require **training data or simulations**.

- **Linearity -** Green's functions require linearity, while DeepONets can approximate solutions for nonlinear equations.
- **Variable Boundary Conditions** – Green's functions require a new derivation for each set of conditions, while DeepONets can generalize across different boundaries.
- **Irregular and Complex Geometries** – Green's functions are often limited to simple shapes, whereas DeepONets can learn solutions in **arbitrary domains**.
- **High-Dimensional Problems** – Green's functions are difficult to compute for **3D and high-dimensional PDEs**, while DeepONets scale more efficiently.
- **Hybrid Data-Driven and Physics-Based Learning** – Green's functions need explicit physics, but DeepONets can **combine physics with real-world data**.

# Neural Operators

1 **Nonlinear PDEs –** Green's functions rely on **linearity**, and DeepONets struggle with **generalizing across nonlinear interactions**. What about **nonlinear function mappings** ?

2 **New Domains & Geometries –** Green's functions are **derived for specific shapes** (e.g., rectangles, spheres), and DeepONets are trained on **fixed-sized inputs**. What about **arbitrary domains** (without retraining)?

3 **Changing Boundary Conditions –** Green's functions require **re-derivation** for each new boundary condition, and DeepONets need **new training**. What about new **boundary variations?**

4 **High-Dimensional PDEs –** Green's functions involve **expensive integration**, and DeepONets require **training on large datasets**. How can we scale to **high-dimensional systems?**

5 **Data-Scarce Problems –** DeepONets need **large training datasets**, Can we learning continuous function mappings with fewer datapoints?

6 **Multiscale & Complex Physics –** Problems like **turbulence, climate modeling, and coupled physical systems** require models that generalize **across scales and conditions**, which Green's functions and DeepONets struggle with.

# Neural Operators

arXiv > cs > arXiv:2108.08481

**Computer Science > Machine Learning**

*[Submitted on 19 Aug 2021 (v1), last revised 2 May 2024 (this version, v6)]*

## Neural Operator: Learning Maps Between Function Spaces

Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, Anima Anandkumar

| Property \ Model | NNs | DeepONets | Interpolation | Neural Operators |
|---|---|---|---|---|
| Discretization Invariance | ✗ | ✗ | ✓ | ✓ |
| Is the output a function? | ✗ | ✓ | ✓ | ✓ |
| Can query the output at any point? | ✗ | ✓ | ✓ | ✓ |
| Can take the input at any point? | ✗ | ✗ | ✓ | ✓ |
| Universal Approximation | ✗ | ✓ | ✗ | ✓ |

Table 1: Comparison of deep learning models. The first row indicates whether the model is discretization invariant. The second and third rows indicate whether the output and input are a functions. The fourth row indicates whether the model class is a universal approximator of operators. Neural Operators are discretization invariant deep learning methods that output functions and can approximate any operator.

Neural Operators generalize across different geometries and boundary conditions without retraining - a significant advantage for applications like climate modeling and fluid dynamics

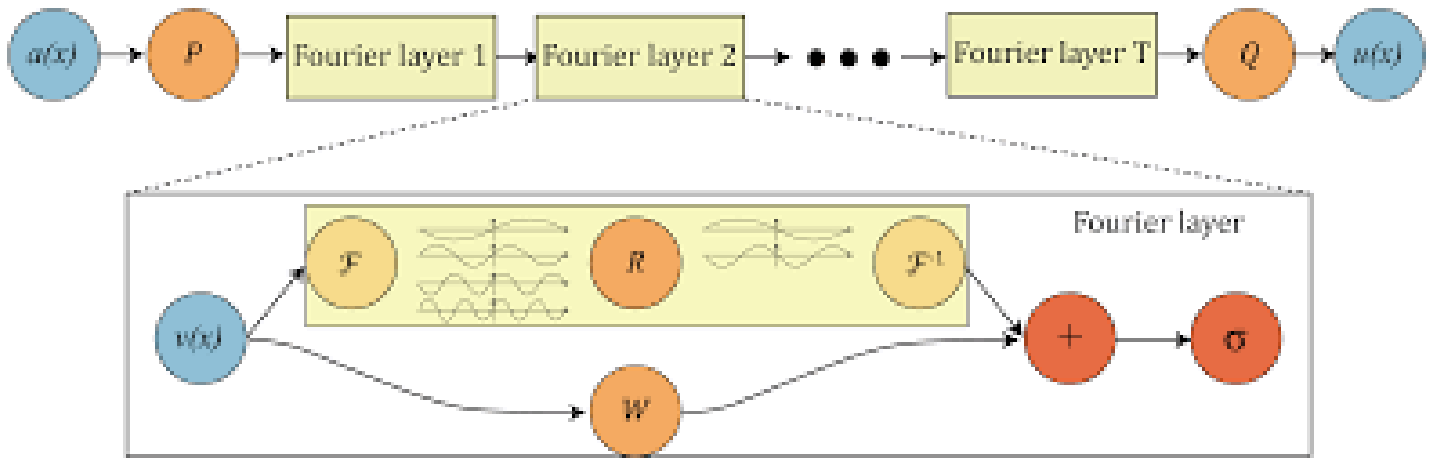
# Neural Operators vs. PINNs and Solvers

| Feature | Neural Operators | PINNs | Classic Solvers |
|---|---|---|---|
| **What is learned?** | Function-to-function mapping | Single PDE solution | Single PDE solution |
| **Training** | Data-driven | Physics-constrained | Not needed |
| **Computational Cost** | Expensive to train, fast inference | Moderate | Expensive |
| **Generalization** | Can generalize across PDE families | Solves one PDE at a time | One equation per simulation |
| **Best for** | High-dimensional PDEs, complex geometries | Data-sparse problems, physics-driven tasks | Benchmarking, high-precision cases |

# Neural Operators: A Peek Below the Hood

Neural Operators aim to approximate **operator mappings** between function spaces. Given an input function f(x) defined over a domain Ω, a Neural Operator learns a mapping: G:f↦u where u(x) is the solution of a differential equation governed by f(x), and G represents the **solution operator**.

Let $D$ be a space of functions defined on a domain Ω, and let $G{:}D{\rightarrow}U$ be an operator mapping from the input function space $D$ (e.g., forcing terms, initial conditions) to the output function space $U$ (e.g., solution of a PDE). For a given function $f$, the corresponding solution is: $u{=}G(f)$. The goal of Neural Operators is to learn $G$ from data.



# Neural Operators: A Peek Below the Hood

**Step 1: Lifting the Input to a High-Dimensional Space**
Define a **lifting operator** PPP that embeds the input function into a higher-dimensional space:
$v_0(x)=P(f(x))$ where $v_0$ is an intermediate representation in an expanded feature space.

**Step 2: Learning the Integral Kernel Operator**
Instead of traditional convolutional layers, Neural Operators use a **global integral kernel** $K_\theta$ to model interactions between function values:

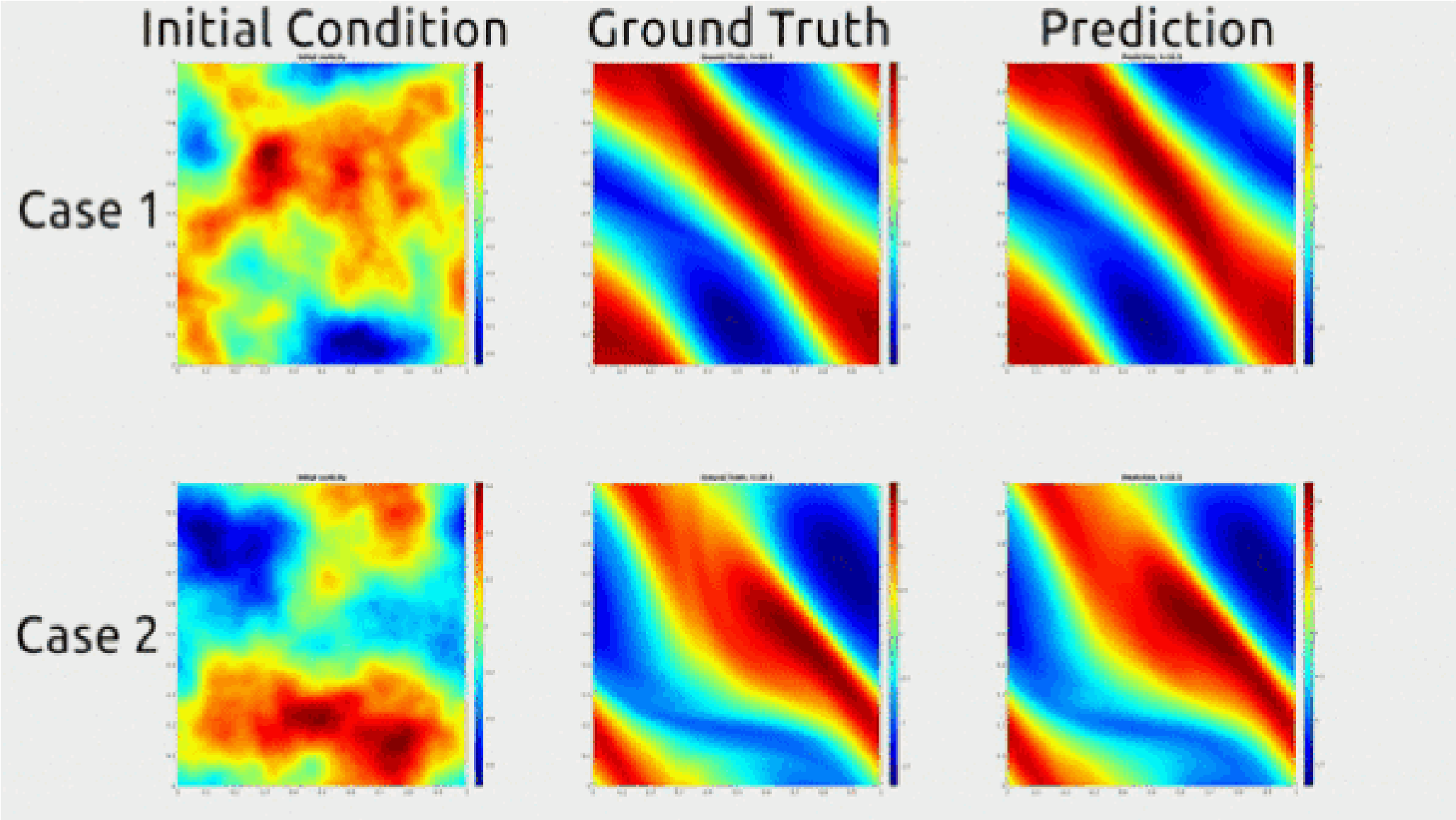
$$v_{k+1}(x)=\sigma\left(\int_\Omega K_\theta(x,x')v_k(x')\,dx'+Wv_k(x)\right)$$

where:
- $K_\theta(x,x')$ is the learned **integral kernel** that encodes relationships between points $x$ and $x'$.
- $W$ is a local transformation (e.g., pointwise linear operator).
- $\sigma$ is a nonlinear activation function.

This equation defines how function values interact across the entire domain, rather than just locally.

**Step 3: Projection Back to Output Space**
After several layers of kernel updates, the final output is obtained by applying a **projection operator** Q:
$u(x)=Q(v_k(x))$
This ensures that the output function remains in the correct solution space.

# Neural Operators Examples




https://zongyi-li.github.io/neural-operator/

| Feature | Green's Functions | DeepONet | Neural Operators |
|---|---|---|---|
| **Problem Type** | Linear differential equations | Data-driven function approximation | Learning operators for PDEs |
| **Boundary Conditions** | Required, explicitly handled | Implicitly learned from data | Can incorporate, but not always explicitly defined |
| **Domain Shape** | Typically simple (e.g., rectangular, spherical) | Can handle irregular domains if trained on them | Works on general domains, flexible input structures |
| **Input Data** | Differential equation and source term | Function values at discrete points | Function values, can handle grid-based or irregular data |
| **Solution Representation** | Integral over Green's function | Neural network-based mapping | Maps functions to functions directly |
| **Training Data Needed?** | No, purely analytical | Yes, needs training data | Yes, needs training data for operator learning |
| **Computational Cost** | High for complex problems, needs explicit integral | Training is costly, but inference is fast | Training is costly, but efficient at inference |
| **Adaptability to New Problems** | Must derive new Green's function for each problem | Can generalize to similar problems if trained properly | Highly generalizable to different PDEs and domains |
| **Memory Usage** | Large for storing Green's functions in high dimensions | Moderate, depends on network size | More efficient for large-scale problems |